LAPORAN PRAKTIKUM UTS

SISTEM DETEKSI DAN REKOMENDASI BURNOUT MAHASISWA

MATA KULIAH SISTEM BERBASIS PENGETAHUAN

Disusun oleh Kelompok 5:

Alvi Choirinnikmah	2341760191
Ismi Atika	2341760036
Muhammad Eka Saputra	2341760021
Muhammad Toyfur D.R	2241760126

JURUSAN TEKNOLOGI INFORMASI
PROGRAM STUDI D-IV SISTEM INFORMASI BISNIS
POLITEKNIK NEGERI MALANG

2026/2027

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Burnout adalah kondisi kelelahan fisik dan mental yang cukup sering dialami oleh mahasiswa, terutama mahasiswa di bidang Teknologi Informasi. Hal ini biasanya disebabkan oleh banyaknya tugas, tekanan deadline, serta kebiasaan begadang untuk menyelesaikan tugas praktikum.

Mahasiswa Teknologi Informasi cenderung memiliki aktivitas belajar yang lebih intens, seperti coding, pengerjaan project, dan praktikum. Kondisi tersebut membuat risiko burnout menjadi lebih tinggi dibandingkan bidang lain. Oleh karena itu, diperlukan sebuah sistem yang dapat membantu mendeteksi tingkat burnout sejak dini serta memberikan rekomendasi penanganannya.

Pada praktikum ini, akan dilakukan analisis beberapa metode, yaitu Forward Chaining, Backward Chaining, Teorema Bayes, dan Certainty Factor. Untuk mengetahui model mana yang sesuai dengan studi kasus project PBL yang akan digunakan sebagai acuan pembuatan sistem berbasis pengetahuan deteksi dan rekomendasi burnout mahasiswa.

1.2 Rumusan Masalah

1. Bagaimana cara melakukan processing dataset burnout agar siap digunakan?
2. Bagaimana implementasi metode Forward Chaining, Backward Chaining, Teorema Bayes, dan Certainty Factor?
3. Metode mana yang paling cocok untuk mendeteksi burnout mahasiswa TI?

1.3 Tujuan

1. Mengolah dataset burnout (150.000 data, 20 filter) menjadi data yang siap digunakan dalam sistem berbasis pengetahuan.
2. Menerapkan empat metode inferensi dalam sistem berbasis pengetahuan untuk mendeteksi tingkat burnout.
3. Menentukan metode terbaik berdasarkan hasil analisis akurasi, precision, recall, dan F1-score

BAB 2 DATASET & PREPROCESSING

2.1 Deskripsi Dataset

Dataset yang digunakan adalah *Student Mental Health and Burnout Dataset* dari Kaggle. Dataset ini berisi informasi terkait kondisi akademik, psikologis, dan gaya hidup siswa. Data ini berfungsi sebagai basis pengetahuan (*Knowledge Base*) awal untuk mengidentifikasi pola-pola yang memicu burnout pada mahasiswa.

Atribut	Keterangan
Sumber	Kaggle - Student Mental Health and Burnout Dataset
Jumlah Data	150.000 baris
Jumlah Fitur	20 kolom (termasuk ID dan label target)
Label Target	Burnout_level

2.2 Penyesuaian Dataset ke Burnout

Dataset diatas masih bersifat umum untuk deteksi burnout mahasiswa teknologi informasi, sehingga dilakukan proses transformasi agar relevan dengan logika sistem deteksi burnout.

1. Pemilihan 8 fitur inti yang paling relevan dengan burnout mahasiswa: stress_level, anxiety_score, depression_score, academic_pressure_score, sleep_quality, daily_study_hours, screeen_time_hours, internet_quality, dan burnout_level.
2. Penambahan 6 variabel turunan khusus TI yang diturunkan dari kombinasi fitur yang ada: coding_hours, debugging_stress, tugas_deadline, project_load, praktikum_preassure, dan screen_time.
3. Transformasi nilai numerik dan kategorikal menjadi fakta-fakta simbolik yang digunakan oleh mesin inferensi untuk Forward Chaining dan Backward Chaining.

2.3 Preprocessing Data

Sebelum data digunakan dalam sistem, dilakukan serangkaian tahapan preprocessing untuk menjamin kualitas data:

1. Penghapusan duplikat menggunakan drop_duplicate() untuk memastikan integritas data.
2. Penanganan missing values: meskipun tidak ditemukan missing values pada dataset, penanganan tetap diimplementasikan untuk memastikan dataset benar-benar bersih sebelum memasuki tahapan berikutnya.

3. Untuk membangun *rule based* yang kuat, dilakukan pemilihan fitur yang relevan dan signifikan terhadap burnout mahasiswa.
   - Kode Preprocessing

Preprocessing Data

[191]
✓ 1d

```python
# Hapus duplikat
df = df.drop_duplicates()

# Missing value
num_cols = df.select_dtypes(include=['int64','float64']).columns
cat_cols = df.select_dtypes(include=['object']).columns

for col in num_cols:
    df[col] = df[col].fillna(df[col].median())

for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

# Bersihkan string
for col in cat_cols:
    df[col] = df[col].astype(str).str.strip().str.lower()

# Outlier handling
df['daily_study_hours'] = df['daily_study_hours'].clip(0, 16)
df['screen_time_hours'] = df['screen_time_hours'].clip(0, 20)
df['anxiety_score'] = df['anxiety_score'].clip(0, 10)
df['depression_score'] = df['depression_score'].clip(0, 10)
df['academic_pressure_score'] = df['academic_pressure_score'].clip(0, 10)
```

   - Kode Pemilihan Variabel + Variabel TI

Pilih Variabel Burnout + TI

[192]
0 d

```python
# ==============================
# SELECTED VARIABLES (ASLI DATASET)
# ==============================
SELECTED_VARS = [
    'stress_level',
    'anxiety_score',
    'depression_score',
    'academic_pressure_score',
    'sleep_quality',
    'daily_study_hours',
    'screen_time_hours',
    'internet_quality',
    'burnout_level'
]

df_selected = df[SELECTED_VARS].copy()


# ==============================
# NORMALISASI
# ==============================
df_selected['stress_level'] = df_selected['stress_level'].str.lower()
df_selected['sleep_quality'] = df_selected['sleep_quality'].str.lower()
df_selected['internet_quality'] = df_selected['internet_quality'].str.lower()
df_selected['burnout_level'] = df_selected['burnout_level'].str.lower()


# ============================================================
# TAMBAHAN VARIABEL KHUSUS MAHASISWA TI (GENERATED)
# ============================================================

# Coding hours (proxy dari study + screen time)
df_selected['coding_hours'] = (
    df_selected['daily_study_hours'] * 0.6 +
    df_selected['screen_time_hours'] * 0.4
)

# Debugging stress (dari anxiety + academic pressure)
df_selected['debugging_stress'] = (
    df_selected['anxiety_score'] + df_selected['academic_pressure_score']
) / 2

# Deadline pressure
df_selected['tugas_deadline'] = df_selected['academic_pressure_score'].apply(
    lambda x: 'high' if x >= 7 else 'low'
)

# Project load
df_selected['project_load'] = df_selected['daily_study_hours'].apply(
    lambda x: 'high' if x > 7 else 'low'
)

# Praktikum pressure
df_selected['praktikum_pressure'] = df_selected['academic_pressure_score'].apply(
    lambda x: 'high' if x >= 7 else 'low'
)

# Screen time kategori
df_selected['screen_time'] = df_selected['screen_time_hours'].apply(
    lambda x: 'high' if x > 8 else 'low'
)


print("Dataset setelah tambahan variabel TI:")
display(df_selected.head())
```

Hasil

```
... Dataset setelah tambahan variabel TI:
```

	stress_level	anxiety_score	depression_score	academic_pressure_score	sleep_quality	daily_study_hours	screen_time_hours	internet_quality	burnout_level	coding_hours	debugging_stress	tugas_deadline	project_load	praktikum_pressure
0	high	10	3	4	average	4.3	6.1	good	high	5.02	7.0	low	low	low
1	high	2	10	8	poor	1.4	3.0	poor	low	2.04	5.0	high	low	high
2	low	2	7	8	good	3.7	1.5	good	high	2.82	5.0	high	low	high
3	high	3	3	4	poor	1.6	7.0	good	high	3.76	3.5	low	low	low
4	high	7	7	6	good	2.0	5.4	good	low	3.36	6.5	low	low	low

- Transformasi Fakta

Tranformasi Fakta

[193]
✓ 5 d

```python
def transform_to_facts(row):
    facts = set()

    # ===== FIX: pakai angka, bukan kolom yang tidak ada =====
    if row['stress_level'] in ['high']:
        facts.add('stress_tinggi')

    if row['anxiety_score'] >= 7:
        facts.add('cemas')

    if row['depression_score'] >= 7:
        facts.add('depresi')

    if row['sleep_quality'] in ['poor']:
        facts.add('tidur_buruk')

    if row['academic_pressure_score'] >= 7:
        facts.add('tekanan_akademik')

    if row['daily_study_hours'] > 8:
        facts.add('belajar_berlebihan')

    # ===== TI =====
    if row['screen_time'] == 'high':
        facts.add('screen_time_tinggi')

    if row['coding_hours'] > 6:
        facts.add('coding_berlebihan')

    if row['debugging_stress'] >= 7:
        facts.add('stress_debugging')

    if row['tugas_deadline'] == 'high':
        facts.add('deadline_tinggi')

    if row['project_load'] == 'high':
        facts.add('banyak_project')

    if row['internet_quality'] == 'poor':
        facts.add('internet_buruk')

    if row['praktikum_pressure'] == 'high':
        facts.add('tekanan_praktikum')

    return facts

df_selected['fakta'] = df_selected.apply(transform_to_facts, axis=1)
```

- Rule Knowledge Base

Rule Knowladge Base Burnout + TI

[194]
✓ 0 d

```python
class Rule:
    def __init__(self, name, conditions, conclusion, logic='AND', cf=1.0):
        self.name = name
        self.conditions = conditions
        self.conclusion = conclusion
        self.logic = logic
        self.cf = cf

    def is_applicable(self, wm):
        if self.logic == 'AND':
            return all(c in wm for c in self.conditions)
        elif self.logic == 'OR':
            return any(c in wm for c in self.conditions)

GOAL = 'risiko_burnout'

KNOWLEDGE_BASE = [
    Rule('R1', ['stress_tinggi', 'cemas'], 'tekanan_mental_tinggi'),
    Rule('R2', ['depresi', 'tidur_buruk'], 'kondisi_psikologis_buruk'),

    Rule('R3', ['tekanan_akademik'], 'tekanan_akademik_tinggi'),
    Rule('R4', ['belajar_berlebihan'], 'kelelahan_akademik'),

    Rule('R5', ['coding_berlebihan', 'stress_debugging'], 'kelelahan_teknis'),
    Rule('R6', ['deadline_tinggi', 'banyak_project'], 'tekanan_project'),
    Rule('R7', ['internet_buruk', 'tekanan_praktikum'], 'hambatan_teknis'),

    # FIX: AND (biar nggak over-detect)
    Rule('R8', ['tekanan_mental_tinggi', 'kondisi_psikologis_buruk'], 'risiko_burnout'),
    Rule('R9', ['tekanan_akademik_tinggi', 'kelelahan_akademik'], 'risiko_burnout'),
    Rule('R10', ['kelelahan_teknis', 'tekanan_project'], 'risiko_burnout'),

    Rule('R11', ['hambatan_teknis'], 'risiko_burnout'),

    Rule('R12', ['risiko_burnout'], 'burnout')
]

GOAL = 'burnout'
```

[195]
✓ 0 d

```python
df_selected['actual'] = df_selected['burnout_level'].apply(
    lambda x: 1 if x in ['high', 'very high'] else 0
)
```

BAB 3 IMPLEMENTASI MODEL

3.1 Forward Chaining

Forward Chaining merupakan metode yang bekerja dari fakta kesimpulan. Contohnya, jika mahasiswa memiliki jam belajar tinggi dan waktu tidur rendah, maka sistem akan menyimpulkan bahwa mahasiswa tersebut mengalami burnout tinggi. Metode ini cukup sederhana dan mudah dipahami, namun kurang fleksibel jika kondisi yang dianalisis cukup kompleks.

- Kode implementasi

a. Forward Chaining

[196]
2d

```
def forward_chaining(facts, rules, goal):
    wm = set(facts)

    while True:
        added = False
        for rule in rules:
            if rule.conclusion not in wm and rule.is_applicable(wm):
                wm.add(rule.conclusion)
                added = True

        if not added or goal in wm:
            break

    return 1 if goal in wm else 0


df_selected['fc'] = df_selected['fakta'].apply(
    lambda f: 1 if forward_chaining(f, KNOWLEDGE_BASE, GOAL) else 0
)
```

3.2 Backward Chaining

Backward Chaining merupakan metode inferensi atau pelacakan dalam kecerdasan buatan (AI) dan sistem pakar yang beroperasi secara goal-driven (berbasis tujuan). Pendekatan ini dimulai dengan hipotesis atau tujuan akhir, kemudian bergerak mundur menggunakan aturan (*rules*) untuk mencari fakta-fakta pendukung yang valid.

- Kode Implementasi

b. Backward Chaining

[197]
✓ 1d

```python
def backward_chaining(goal, facts, rules):
    if goal in facts:
        return True

    for rule in rules:
        if rule.conclusion == goal:
            if all(backward_chaining(c, facts, rules) for c in rule.conditions):
                return True

    return False


df_selected['bc'] = df_selected['fakta'].apply(
    lambda f: 1 if backward_chaining(GOAL, f, KNOWLEDGE_BASE) else 0
)
```

3.3 Teorema Bayes

Teorema Bayes merupakan metode probabilistik yang menghitung suatu peluang hipotesis berdasarkan bukti yang ada. contohnya, jika mahasiswa memiliki skor anxiety yang tinggi dan depression score tinggi, maka sistem akan menghitung probabilitas mahasiswa tersebut mengalami burnout. metode ini belajar langsung dari data, namun mengasumsikan setiap fitur bersifat independen satu sama lain yang belum tentu terpenuhi di dunia nyata.

- Kode Implementasi

c. Teorema Bayes

[198]
✓ 0d

```python
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

features = [
    'anxiety_score',
    'depression_score',
    'academic_pressure_score',
    'daily_study_hours',
    'screen_time_hours'
]

X = df_selected[features]
y = df_selected['actual']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

nb = GaussianNB()
nb.fit(X_train, y_train)

df_selected['nb'] = nb.predict(X)
```

3.4 Certainty Factor

Merupakan metode inferensi dalam kecerdasan buatan (AI) dan sistem pakar yang digunakan untuk menangani ketidakpastian dalam dalam proses pengambilan keputusan. Metode ini menyatakan tingkat keyakinan terhadap suatu hipotesis berdasarkan fakta atau gejala yang ada, dengan nilai antara -1 (tidak percaya) hingga 1 (sangat percaya).

- Kode Implementasi

d. Certainty Factor

[199] ✓ 0 d

```
KNOWLEDGE_BASE = [

    # ===== MENTAL (PALING KUAT) =====
    Rule('R1', ['stress_tinggi', 'cemas'], 'tekanan_mental_tinggi', cf=0.85),
    Rule('R2', ['depresi', 'tidur_buruk'], 'kondisi_psikologis_buruk', cf=0.85),

    # ===== AKADEMIK (KUAT) =====
    Rule('R3', ['tekanan_akademik'], 'tekanan_akademik_tinggi', cf=0.7),
    Rule('R4', ['belajar_berlebihan'], 'kelelahan_akademik', cf=0.65),

    # ===== TEKNIS (SEDANG) =====
    Rule('R5', ['coding_berlebihan', 'stress_debugging'], 'kelelahan_teknis', cf=0.7),
    Rule('R6', ['deadline_tinggi', 'banyak_project'], 'tekanan_project', cf=0.7),
    Rule('R7', ['internet_buruk', 'tekanan_praktikum'], 'hambatan_teknis', cf=0.6),

    # ===== KOMBINASI (SANGAT KUAT) =====
    Rule('R8', ['tekanan_mental_tinggi', 'kondisi_psikologis_buruk'], 'risiko_burnout', cf=0.9),
    Rule('R9', ['tekanan_akademik_tinggi', 'kelelahan_akademik'], 'risiko_burnout', cf=0.75),
    Rule('R10', ['kelelahan_teknis', 'tekanan_project'], 'risiko_burnout', cf=0.75),

    # ===== KHUSUS =====
    Rule('R11', ['hambatan_teknis'], 'risiko_burnout', cf=0.6),

    # ===== FINAL =====
    Rule('R12', ['risiko_burnout'], 'burnout', cf=1.0)
]
```

[200] ✓ 0 d

```
def combine_cf(cf1, cf2):
    return cf1 + cf2 * (1 - cf1)


def certainty_factor(facts, KB_CF):
    cf_values = {}

    for rule in KB_CF:
        if all(c in facts for c in rule.conditions):
            cf_values[rule.conclusion] = rule.cf

    return cf_values.get('risiko_burnout', 0)   # ✓ FIX DI SINI
```

BAB 4 ANALISIS & PERBANDINGAN MODEL

4.1 Hasil Setiap Model

- Hasil

Hasil Model

```
df_hasil = pd.DataFrame([
    ['Forward Chaining', *fc_eval.values()],
    ['Backward Chaining', *bc_eval.values()],
    ['Naive Bayes', *nb_eval.values()],
    ['Certainty Factor', *cf_eval.values()]
], columns=['Model','Accuracy','Precision','Recall','F1 Score'])

display(df_hasil.sort_values(by='F1 Score', ascending=False))
```

	Model	Accuracy	Precision	Recall	F1 Score
0	Forward Chaining	0.595307	0.556820	0.595307	0.569079
1	Backward Chaining	0.595307	0.556820	0.595307	0.569079
3	Certainty Factor	0.668227	0.446527	0.668227	0.535331
2	Naive Bayes	0.476100	0.556146	0.476100	0.499456

4.2 Perbandingan Model

Model	Deskripsi
Forward Chaining	Memiliki performa paling seimbang dengan F1 Score tertinggi, cocok untuk hasil yang stabil antara precision dan recall.
Backward Chaining	Performa identik dengan forward chaining, menunjukkan tidak ada perbedaan signifikan dalam hasil inferensi.
Certainty Factor	Memiliki accuracy dan recall tertinggi, lebih sensitif dalam mendeteksi kasus. tetapi precision rendah sehingga banyak false positive.
Teorema Bayes	Performa paling rendah dibanding model lain, kurang optimal tanpa perbaikan data atau fitur.

4.3 Analisis Mendalam

1. Forward Chaining

   Forward Chaining menunjukkan performa paling seimbang dengan F1-score tertinggi (0.5691). Hal ini menandakan model mampu menjaga keseimbangan antara precision dan recall, sehingga tidak terlalu banyak menghasilkan kesalahan baik *false positive*

maupun *false negative*. Karena berbasis rule, model ini juga mampu merepresentasikan hubungan sebab-akibat antara gejala burnout secara logis, sehingga cocok untuk sistem diagnosis yang membutuhkan interpretasi yang jelas dan konsisten.

2. Backward Chaining

   Backward Chaining memiliki hasil yang identik dengan Forward Chaining pada semua metrik. Hal ini terjadi karena keduanya menggunakan basis aturan yang sama. Namun secara pendekatan, Backward Chaining bersifat *goal-driven* sehingga kurang optimal ketika sistem menerima input berupa gejala dari pengguna. Oleh karena itu, meskipun performanya sama, penggunaanya tidak sefleksibel Forward Chaining dalam konteks sistem deteksi.

3. Certainty Factor

   Certainty Factor memiliki nilai accuracy dan recall tertinggi yaitu (0.6682), yang menunjukkan bahwa model sensitif dalam mendeteksi kasus burnout dan mampu menangkap lebih banyak kondisi yang berpotensi beresiko. Namun, memiliki nilai precision yang rendah (0.4465) mengindikasikan adanya *false positive* yang dapat menurunkan kepercayaan user.

4. Teorema Bayes

   Teorema Bayes memiliki hasil performa yang rendah dengan F1-score sebesar 0.4995, yang disebabkan oleh independensi antar fitur yang tidak sesuai dengan data burnout. Akibatnya model tidak bisa menangkap pola yang kompleks dalam data, sehingga akurasi dan klasifikasinya kurang optimal.

BAB 5 PEMILIHAN MODEL TERBAIK

5.1 Model Terpilih

Berdasarkan analisis yang telah kami lakukan dari keempat metode terhadap karakteristik dataset *Student Mental Health and Burnout*, model yang terpilih sebagai model terbaik adalah Forward Chaining. Metode Forward Chaining ini menunjukkan performa yang identik dengan nilai F1-Score sebesar 0.569079.

5.2 Alasan Akademis

Pemilihan model didasarkan dari perolehan hasil pada matrik evaluasi serta karakteristik logika sistem sebagai berikut:

1. Optimalisasi F1-score: meskipun Certainty Factor unggul dalam akurasi, metode Forward dipilih karena memiliki F1-score tertinggi 0.569079. Hal ini menunjukkan keseimbangan performa antara precision dan recall, sehingga model lebih stabil dan akurat dalam mengklasifikasikan risiko burnout tanpa bias pada satu metrik saja.
2. Kesesuaian Karakteristik Data: dataset memiliki fitur yang bersifat sekuensial, sehingga metode berbasis chaining lebih efektif memetakan alur logika sebab akibat antar gejala. Hal ini berbeda dengan Naive Bayes yang mengasumsikan setiap gejala bersifat independen, sehingga gagal menangkap keterkaitan antar variabel yang krusial dalam diagnosis kesehatan mental.
3. Interpretabilitas Model: forward chaining memikeunggulan dalam hal intertabilitas karena proses pengambilan keputusannya didasarkan pada aturan (*rule-based*). Hal ini penting dalam sistem diagnosis, terutama pada bidang kesehatan mental, karena hasil diagnosis dapat dijelaskan secara logis kepada pengguna.
4. Konsistensi Hasil Inferensi: Hasil evaluasi menunjukkan bahwa Forward dan Backward Chaining memiliki performa yang identik. Namun, Forward Chaining lebih sesuai digunakan karena pendekatannya berbasis data awal (data-driven), sehingga lebih konsisten dalam sistem yang memulai analisis dari gejala yang diberikan oleh pengguna.
5. Trade-off Precision dan Recall: certainty factor memiliki recall yang tinggi, namun precision yang rendah menunjukkan adanya kecenderungan (*false positive*). Model dengan kecenderungan (*false positive*) dapat menurunkan kepercayaan pengguna.

BAB 6 KESIMPULAN & SARAN

6.1 Kesimpulan

Proyek ini berhasil mengimplementasikan keempat metode Sistem Berbasis Pengetahuan: Forward Chaining, Backward Chaining, Teorema Bayes, dan Certainty Factor. pada dataset *Student Mental Health and Burnout* yang terdiri dari 150.000 data mahasiswa. Setelah melalui proses preprocessing yang komprehensif (penghapusan duplikat, pemilihan fitur, dan penambahan variabel turunan khusus mahasiswa TI) keempat model berhasil berjalan dengan baik dan menghasilkan deteksi burnout yang relevan.

Forward Chaining dipilih sebagai model terbaik karena memiliki F-1 Score tertinggi sebesar 0.569079, menunjukkan keseimbangan yang paling stabil antara precision dan recall dibandingkan model lainnya. Model ini paling sesuai untuk studi kasus deteksi burnout mahasiswa TI yang membutuhkan klasifikasi yang konsisten dan mudah diinterpretasikan.

6.2 Saran

Berdasarkan pada hasil penelitian, terdapat saran untuk pengembangan selanjutnya. Pertama, *rule knowledge base* dapat diperluas dan divalidasi dengan melibatkan pakar agar lebih akurat. Kedua, dataset dapat diperkaya dengan variabel yang lebih relevan untuk meningkatkan kualitas prediksi. Ketiga, penelitian selanjutnya dapat mengeksplorasi metode yang lebih kompleks seperti model hybrid. Keempat, sistem dapat dikembangkan menjadi aplikasi berbasis web atau mobile agar lebih mudah digunakan.

Link Repositori Github:
https://github.com/EksaMuhammad/UTS-SBP-Kel-5

Link google collab:
https://colab.research.google.com/drive/1H3OS2uekc1Cimt7zGL8yNSLSoWzHNpSp?usp=sharing

Link Dataset:
https://www.kaggle.com/datasets/sehaj1104/student-mental-health-and-burnout-dataset